

CHAPTER IV

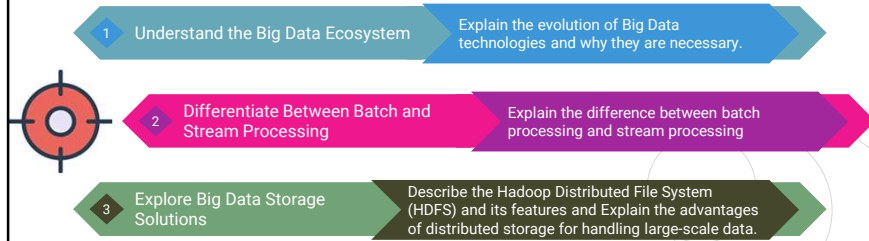
Map Reduce Programming Model

1. BIG DATA WITH TRADITIONAL DATA STORAGE
2. BIG DATA ECOSYSTEM
3. STORAGE & PROCESSING
3. HDFS ARCHITECTURE & DATA STORAGE

Big Data Ecosystem & Technologies

Section 01

Objectives



Section N° 1

HADOOP MAP REDUCE

Hadoop Map Reduce

Presentation

Section 01



MapReduce is a programming model and distributed computing framework for processing large datasets in parallel across a cluster of computers.

Hadoop MapReduce is the open-source version of MapReduce which is released within the Apache Hadoop project.



Sanjay Ghemawat



Jeffrey Dean

They led the conception, design, and implementation of much of Google's revolutionary software infrastructure, which underlies the company's web search and indexing, as well as numerous applications across the industry.

The 2004 paper "MapReduce: Simplified Data Processing on Large Clusters" is the seminal article on MapReduce. The article describes the methods used to split, process, and aggregate the large amount of data for the Google search engine.

Hadoop Map Reduce Characteristics

Section 01



Parallel Processing	- Processes large datasets by distributing tasks across machines. - Improves efficiency by executing tasks in parallel.
Scalability	- Can handle petabytes of data by adding more nodes to the cluster. - Works well in distributed computing environments like Hadoop.
Fault Tolerance	- Automatically handles node failures by reassigning tasks. - Data is replicated to ensure reliability.
Data Locality Optimization	- Processes data close to its storage location to reduce network traffic.
Simplified Programming Model	- Abstracts complex distributed processing. - Uses key-value pairs for data representation.
Batch Processing	- Suited for large-scale batch jobs rather than real-time processing. - Works well for log analysis, indexing, and data aggregation.

Fouad DAHAK
ESIA, 2024/2025

Hadoop Map Reduce Parallel Processing

Section 01

Parallel Processing

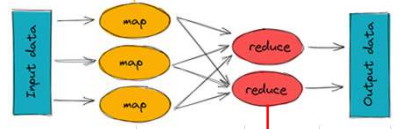
- ❖ Task is broken up to multiple parts with a software tool
- ❖ and each part is distributed to a processor,
- ❖ then each processor will perform the assigned part.
- ❖ Finally, the parts are reassembled to deliver the final solution or execute the task.

Programming Model

- ❖ MapReduce works by breaking the processing into two phases: map and reduce.
- ❖ Each phase uses key-value pairs as input and output, the types of which can be chosen by the user.

map (Inkey, InValue) - list (Outkey, InterValue)

- Processes input key/value pair
- One Map()-Call for every pair
- Produces a set of intermediate pairs



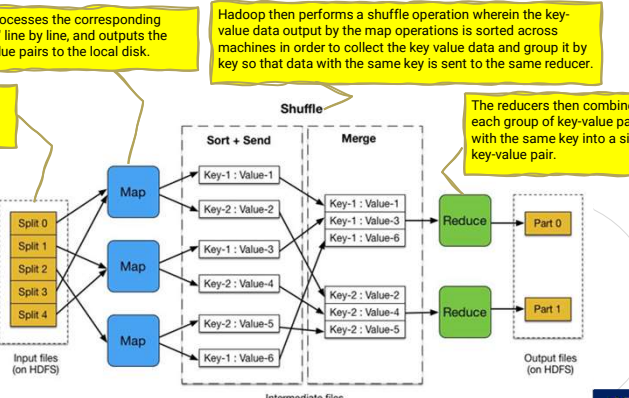
reduce (OutKey, list(InterValue)) - list (OutValue)

- Combines all intermediate values for a particular key;
- One Reduce()-Call per unique key
- Produces a set of merged output values (usually just one output value)

Fouad DAHAK
ESIA, 2024/2025

Hadoop Map Reduce Paradigm

Section 01



Each mapper processes the corresponding input file "splits" line by line, and outputs the resulting key-value pairs to the local disk.

The input files are split and fed to mappers on different machines.

Since the shuffle phase is carried out automatically by Hadoop, the user only needs to define the map and reduce operations.

Hadoop then performs a shuffle operation wherein the key-value data output by the map operations is sorted across machines in order to collect the key value data and group it by key so that data with the same key is sent to the same reducer.

The reducers then combine each group of key-value pairs with the same key into a single key-value pair.

Sort + Send Merge

Intermediate files (on local disks)

Output files (on HDFS)

Fouad DAHAK
ESIA, 2024/2025

Hadoop Map Reduce Functional Programming

Section 01

Functional Programming

- ❖ Map and Reduce are based on functional programming
- ❖ (FP) is a programming paradigm that treats computation as the evaluation of mathematical functions and avoids changing state and mutable data.

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class FunctionalExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        List<Integer> squaredNumbers = numbers.stream()
            .map(n -> n * n)
            .collect(Collectors.toList());
        System.out.println(squaredNumbers);
    }
}
```

Step 1: Create a list of numbers

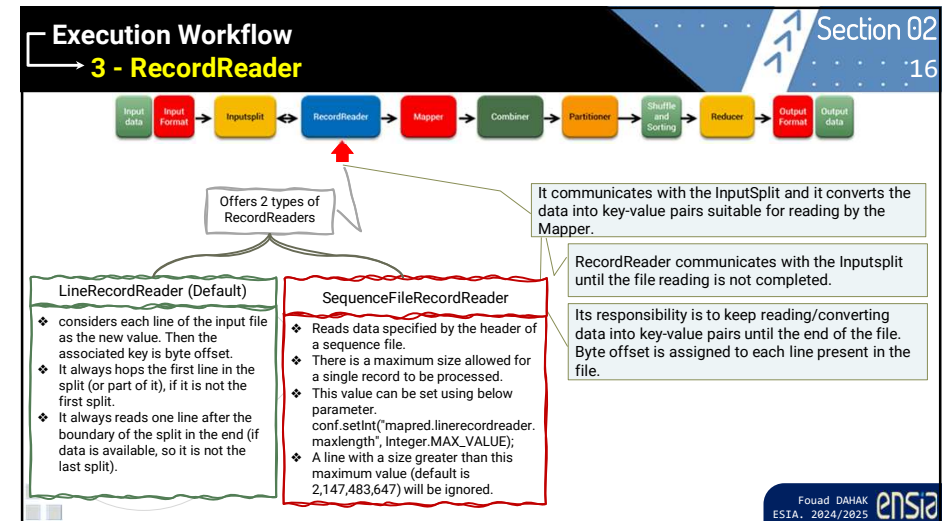
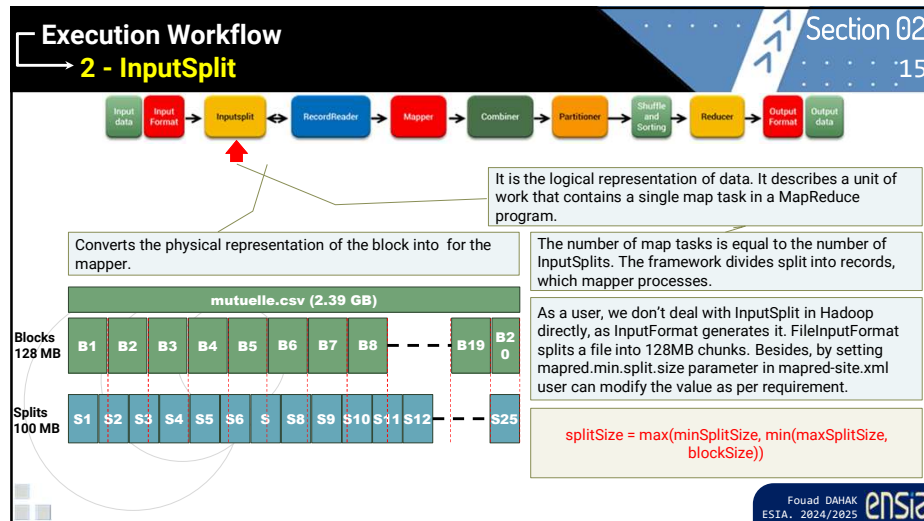
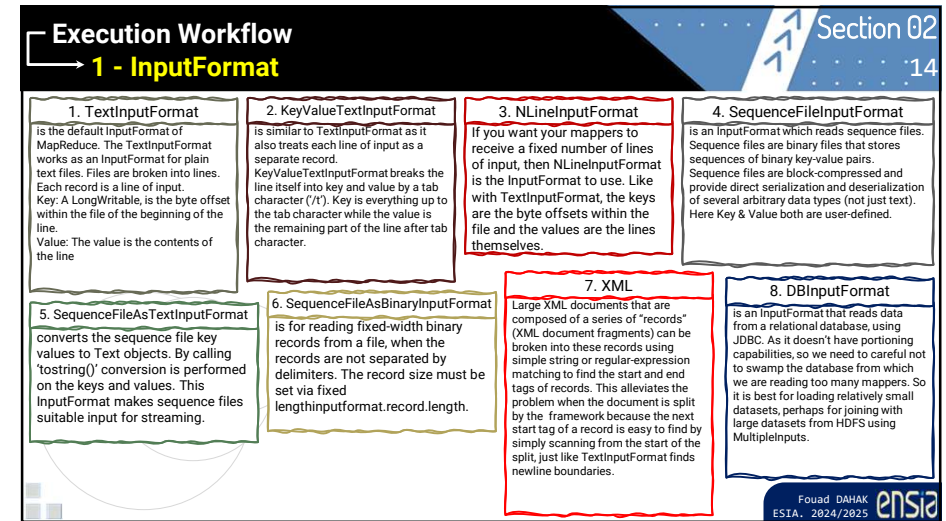
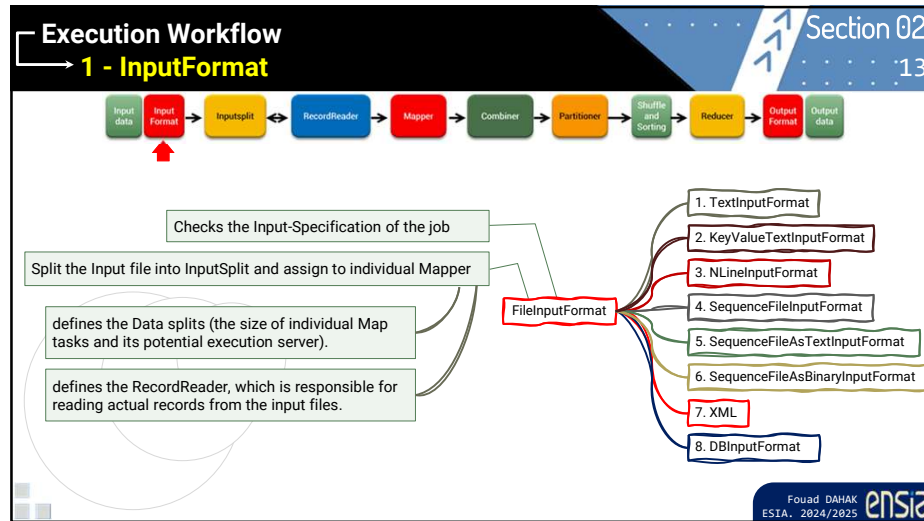
Convert list to stream

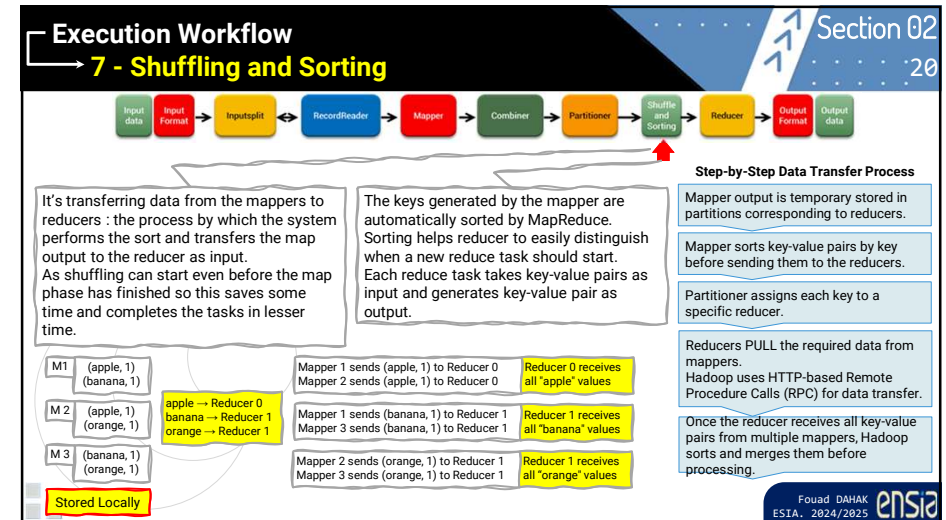
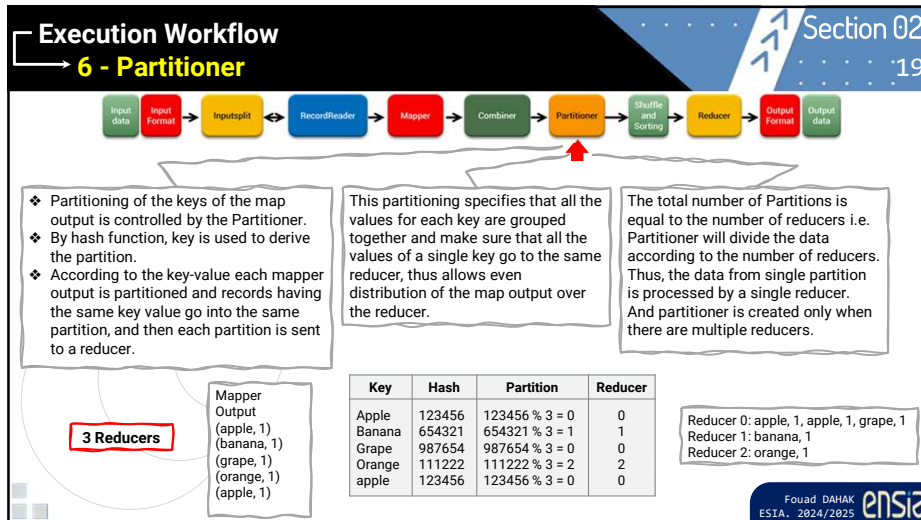
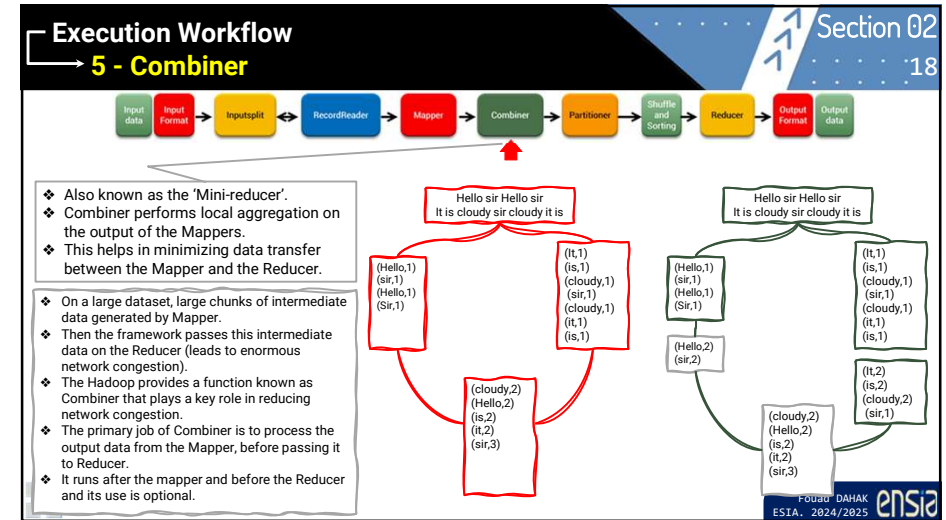
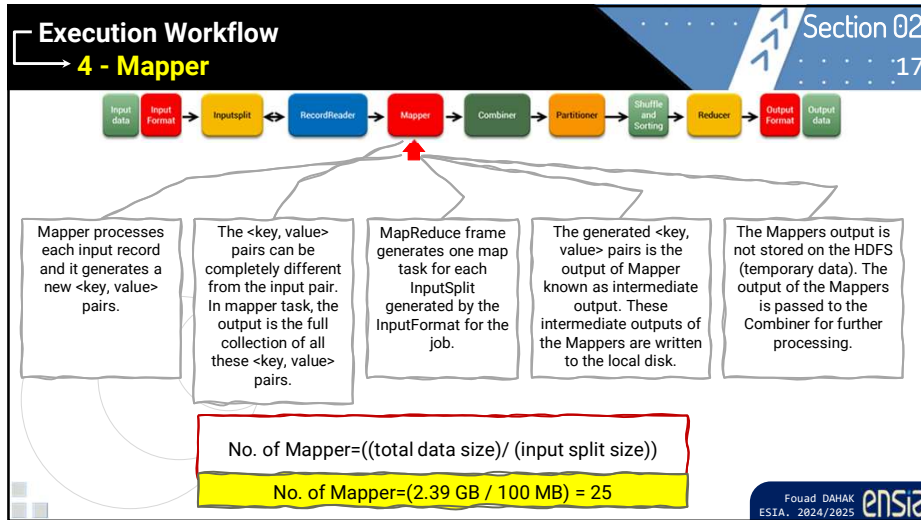
Step 2: Use functional programming (map) to square each number

Convert stream back to list

Step 3: Print the result
Output: [1, 4, 9, 16, 25]

Fouad DAHAK
ESIA, 2024/2025





Execution Workflow

8 - Reducer

Section 02 21

- After processing the data, it produces a new set of output. At last, HDFS stores this output data.
- Reducer takes a set of an intermediate key-value pair produced by the mapper as the input and runs a Reducer function on each of them.
- One can aggregate, filter, and combine this data (key, value) in a number of ways for a wide range of processing.

One-one mapping takes place between keys and reducers. Reducers run in parallel since they are independent of one another. The user decides the number of reducers. By default, number of reducers is 1.

With the help of `Job.setNumreduceTasks(int)` the user set the number of reducers for the job

The right number of reducers are 0.95 or 1.75 multiplied by $(\text{no. of nodes} \times \text{no. of the maximum container per node})$. With 0.95, all reducers immediately launch and start transferring map outputs as the maps finish. With 1.75, the first round of reducers is finished by the faster nodes and second wave of reducers is launched doing a much better job of load balancing.

Fouad DAHAK
ESIA, 2024/2025

Execution Workflow

9 - OutputFormat

Section 02 22

Check the output specification for execution of the Map-Reduce job how RecordWriter is used to write output to output files.

RecordWriter takes output data from Reducer and writes it to output files. The way these output key-value pairs are written in output files is determined by the Output Format. OutputFormat instances are used to write to files on the HDFS or local disk.

- 1. TextOutputFormat
- 2. SequenceFileOutputFormat
- 3. SequenceFileAsBinaryOutputFormat
- 4. MapFileOutputFormat
- 5. MultipleOutputs
- 6. LazyOutputFormat
- 7. DBOutputFormat

Fouad DAHAK
ESIA, 2024/2025

Execution Workflow

9 - OutputFormat

Section 02 23

- 1. TextOutputFormat**
is the default reducer Output Format, which writes (key, value) pairs on individual lines of text files and its keys and values can be of any type since TextOutputFormat turns them to string. Each key-value pair is separated by a tab character.
- 2. SequenceFileOutputFormat**
is an Output Format which writes sequences files for its output and it is intermediate format use between MapReduce jobs, which rapidly serialize arbitrary data types to the file; and the corresponding SequenceFileInputFormat will deserialize the file into the same types and presents the data to the next mapper in the same manner as it was emitted by the previous reducer, since these are compact and readily compressible.
- 3. SequenceFileAsBinaryOutputFormat**
is another variant of SequenceFileInputFormat. It also writes keys and values to sequence file in binary format.
- 4. MapFileOutputFormat**
another form of FileOutputFormat. It writes output as map files. The framework adds a key in a MapFile in order. So we need to ensure that reducer emits keys in sorted order.
- 5. MultipleOutputs**
allows writing data to files whose names are derived from the output keys and values, or in fact from an arbitrary string.
- 6. LazyOutputFormat**
Sometimes FileOutputFormat will create output files, even if they are empty. LazyOutputFormat is a wrapper OutputFormat which ensures that the output file will be created only when the record is emitted for a given partition.
- 7. DBOutputFormat**
is an Output Format for writing to relational databases and HBase. It sends the reduce output to a SQL table. It accepts key-value pairs, where the key has a type extending DBWritable.

Fouad DAHAK
ESIA, 2024/2025

Execution Workflow

Understanding the Key-Value Pairs in Map Reduce

Section 02 24

- 1. Map Input**
Map-input by default will take the line offset as the key and the content of the line will be the value as Text. By using custom InputFormat we can modify them.
- 2. Map Output**
Map basic responsibility is to filter the data and provide the environment for grouping of data based on the key.
Key – It will be the field/ text/ object on which the data has to be grouped and aggregated on the reducer side.
Value – It will be the field/ text/ object which is to be handled by each individual reduce method.
- 3. Reduce Input**
The output of Map is the input for reduce, so it is same as Map-Output.
- 4. Reduce Output**
It depends on the required output.

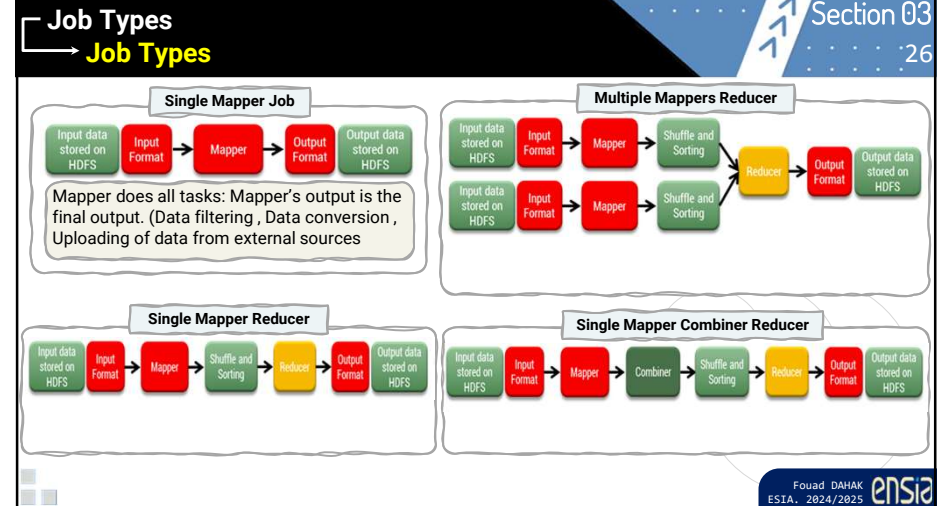
Generation of a key-value pair depends on the dataset and the required output.

Fouad DAHAK
ESIA, 2024/2025

Section N° 3

JOB TYPES & DESIGN PATTERNS

Fouad DAHAK, ENSIA, 2024/2025

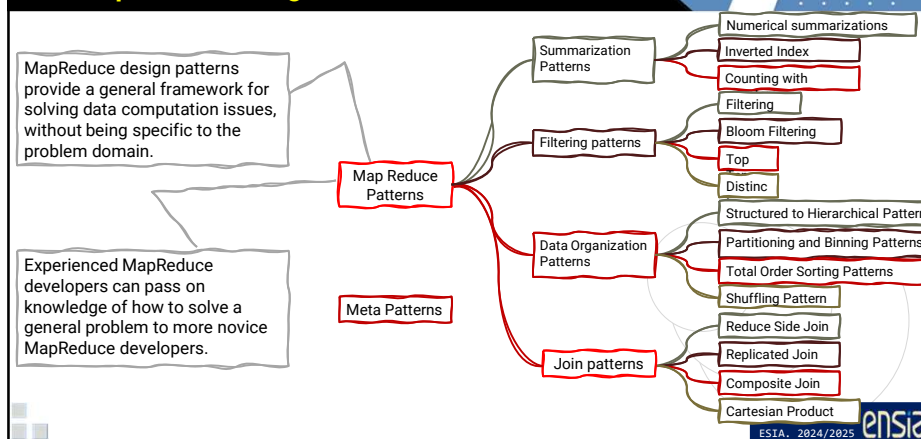


Map Reduce Patterns

Map Reduce Design Patterns

Section 03

27

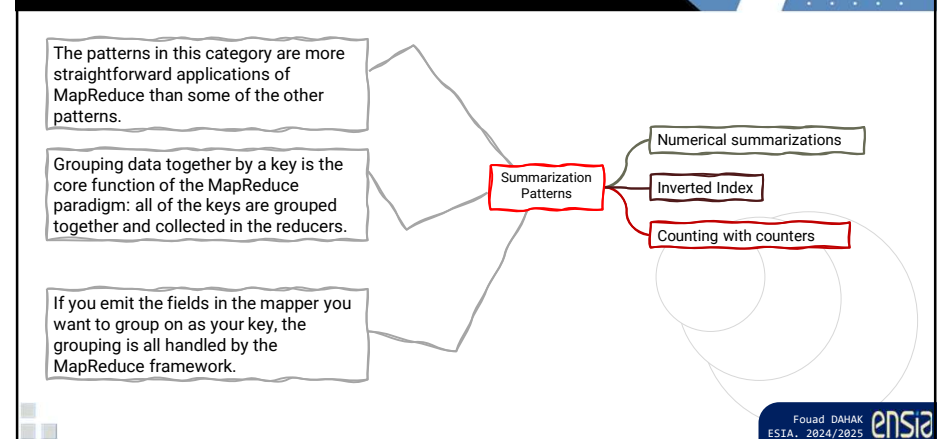


Map Reduce Patterns

Summarization Patterns

Section 03

28



Map Reduce Patterns

Summarization Patterns

Numerical Summarizations

Group records together by a key field and calculate a numerical aggregate per group to get a top-level view of the larger data set.

It should be used when :

- ✓ You are dealing with numerical data or counting.
- ✓ The data can be grouped by specific fields.

Inverted Index Summarizations

Generate an index from a data set to allow for faster searches or data enrichment capabilities.

Counting with Counters

This pattern utilizes the MapReduce framework's counters utility to calculate a global sum entirely on the map side without producing any output.

Numerical summarizations

- Word count
- Record count
- Min/Max/Count
- Average/Median/Standard deviation

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Patterns

Filtering patterns

The patterns in this category all have one thing in common: they don't change the actual records. These patterns all find a subset of data, whether it be small, like a top-ten listing, or large, like the results of a deduplication. This differentiates filtering patterns from those in the previous category, which was all about summarizing and grouping data by similar fields to get a top-level view of the data.

Filtering is more about understanding a smaller piece of your data, such as all records generated from a particular user, or the top ten most used verbs in a corpus of text.

In short, filtering allows you to apply a microscope to your data. It can also be considered a form of search. If you are interested in finding all records that involve a particular piece of distinguishing information, you can filter out records that do not match the search criteria.

Filtering Patterns

- Filtering
- Bloom Filtering
- Top Ten
- Distinct

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Patterns

Filtering patterns

Filtering

As the most basic pattern, filtering serves as an abstract pattern for some of the other patterns. Filtering simply evaluates each record separately and decides, based on some condition, whether it should stay or go.

The data can be parsed into "records" that can be categorized through some well-specified criterion determining whether they are to be kept.

Bloom Filtering

Bloom filtering does the same thing as the previous pattern, but it has a unique evaluation function applied to each record.

- ✓ Data can be separated into records, as in filtering.
- ✓ A feature can be extracted from each record that could be in a set of hot values.
- ✓ There is a predetermined set of items for the hot values.

Top Ten

Top ten pattern is a bit different than previous ones in that you know how many records you want to get in the end, no matter what the input size. In generic filtering, however, the amount of output depends on the data.

- ✓ Requires a comparator function ability between two records.
- ✓ The number of output records should be significantly fewer than the number of input records.

Distinct

This pattern filters the whole set, but it's more challenging because the final output of this filter application is a set of unique records.

You have duplicates values in your data set

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Patterns

Data Organization Patterns

These patterns are all about reorganizing data. The value of individual records is often multiplied by the way they are partitioned, sharded, or sorted. This is especially true in distributed systems, where partitioning, sharding, and sorting can be exploited for performance.

In many organizations, Hadoop and other MapReduce solutions are only a piece in the larger data analysis platform. Data will typically have to be transformed in order to interface nicely with the other systems. Likewise, data might have to be transformed from its original state to a new state to make analysis in MapReduce easier.

These patterns are often used together to solve data organization problems. For example, you may want to restructure your data to be hierarchical, bin the data, and then have the bins be sorted.

Data Organization Patterns

- Structured to Hierarchical Pattern
- Partitioning and Binning Patterns
- Total Order Sorting Patterns
- Shuffling Pattern

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Patterns

Data Organization Patterns

Section 03 33

Structured to Hierarchical

The structured to hierarchical pattern creates new records from data that started in a very different structure. The intent is to transform your row-based data to a hierarchical format, such as JSON or XML.

- You have data sources that are linked by some set of foreign keys.
- Your data is structured and row-based.

Partitioning

The partitioning pattern moves the records into categories (i.e., shards, partitions, or bins) but it doesn't really care about the order of records. The intent is to take similar records in a data set and partition them into distinct, smaller data sets.

knowing how many partitions you are going to have ahead of time

Binning

The binning pattern, much like the previous pattern, moves the records into categories irrespective of the order of records. The intent is for each record in the data set, file each one into one or more categories.

Total Order Sorting

The total order sorting pattern is concerned with the order of the data from record to record.

Shuffling

The total order sorting and shuffling patterns are opposites in terms of effect, but the latter is also concerned with the order of data in records. We use it when we have a set of records that we want to completely randomize.

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Patterns

Join patterns

Section 03 34

Data is all over the place, and while it's very valuable on its own, we can discover interesting relationships when we start analyzing these sets together. This is where join patterns come into play. Joins can be used to enrich data with a smaller reference set or they can be used to filter out or select records that are in some type of special list. The use cases go on and on as well.

As with any MapReduce operation, network bandwidth is a very important resource and joins have a tendency to use a lot of it. Anything we can do to make the network transfer more efficient is worthwhile, and network optimizations are what differentiates these patterns.

```

graph LR
    JP[Join patterns] --> RSJ[Reduce Side Join]
    JP --> RJ[Replicated Join]
    JP --> CJ[Composite Join]
    JP --> CP[Cartesian Product]
  
```

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Patterns

Join patterns

Section 03 35

Reduce Side Join

The reduce side join pattern can take the longest time to execute compared to the other join patterns, but it is simple to implement and supports all the different join operations.

Replicated Join

A replicated join is a special type of join operation between one large and many small data sets that can be performed on the map-side.

- The type of join to execute is an inner join or a left outer join, with the large input data set being the "left" part of the operation.
- All of the data sets, except for the large one, can be fit into main memory of each map task.

Composite Join

A composite join is a specialized type of join operation that can be performed on the map-side with many very large formatted inputs.

Cartesian Product

Cartesian product pattern is an effective way to pair every record of multiple inputs with every other record. A job using this pattern can take an extremely long time to complete.

- You want to analyze relationships between all pairs of individual records.
- You've exhausted all other means to solve this problem.
- You have no time constraints on execution time.

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Patterns

Meta Patterns

Section 03 36

```

graph LR
    MP[Meta Patterns] --> JC[Job chaining]
    MP --> JM[Job Merging]
  
```

When we can't solve a problem with a single Map Reduce job. Some jobs in a chain will run in parallel, some will have their output fed into other jobs, and so on.

Job chaining is one of the more complicated processes to handle because it's not a feature out of the box in most MapReduce frameworks.

Hadoop is designed for handling one MapReduce job very well, but handling a multistage job takes a lot of manual coding.

Job merging is an optimization aimed to reduce the amount of I/O through the MapReduce pipeline..

Job merging is a process that allows two unrelated jobs that are loading the same data to share the MapReduce pipeline.

The main benefit of merging is that the data needs to be loaded and parsed only once.

Fouad DAHAK
ESIA, 2024/2025

Section N° 4

MAP REDUCE APPLICATIONS

Fouad DAHAK, ENSIA, 2024/2025

Map Reduce Applications Job Components

Section 04

38

The MapReduce API is written in Java, so MapReduce applications are primarily Java-based.

The components of a MapReduce application that you can develop

From within the driver, you can use the MapReduce API, which includes factory methods to create instances of all components in the preceding list. (In case you're not a Java person, a factory method is a tool for creating objects.)

A generic API named Hadoop Streaming lets you use other programming languages (most commonly, C, Python, and Perl). Also, you can code mapper and reducer functions only with Hadoop Streaming.

Components

Driver (mandatory)

Mapper class (mandatory)

Reducer class (optional)

Combiner class (optional)

Partitioner class (optional)

RecordReader and RecordWriter (optional)

Fouad DAHAK
ESIA, 2024/2025 ensia

Map Reduce Applications Job Components

Section 04

39

Driver (mandatory)

This is the application shell that's invoked from the client. It configures the MapReduce Job class (which you do not customize) and submits it to the Resource Manager.

Mapper (mandatory)

The Mapper class you implement needs to define the formats of the key/value pairs you input and output as you process each record. This class has only a single method, named `map`, which is where you code how each record will be processed and what key/value to output. To output key/value pairs from the mapper task, write them to an instance of the Context class.

Reducer (optional)

The reducer is optional for map-only applications where the Reduce phase isn't needed.

Combiner (optional)

A combiner can often be defined as a reducer, but in some cases, it needs to be different.

Partitioner (optional)

Customize the default partitioner to perform special tasks, such as a secondary sort on the values for each key or for rare cases involving sparse data and imbalanced output files from the mapper tasks.

RecordReader & RecordWriter (optional)

Fouad DAHAK
ESIA, 2024/2025 ensia

Map Reduce Applications Job Implementation

Section 04

40

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
```

By default, job properties are read from the configuration files stored in `/etc/hadoop/conf`, but you can override them by setting your Job class properties.

```
public class FlightsByCarrier {
    public static void main(String[] args) throws Exception {
        Job job = new Job();
        job.setJarByClass(FlightsByCarrier.class);
        job.setJobName("FlightsByCarrier");

        TextInputFormat.addInputPath(job, new Path(args[0]));
        job.setInputFormatClass(TextInputFormat.class);
        job.setMapperClass(FlightsByCarrierMapper.class);
        job.setReducerClass(FlightsByCarrierReducer.class);

        TextOutputFormat.setOutputPath(job, new Path(args[1]));
        job.setOutputFormatClass(TextOutputFormat.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.waitForCompletion(true);
    }
}
```

The Job class represents the entire MapReduce application.

Using the input path we catch from the main method, we identify the HDFS path for the data to be processed and the expected format of the data.

We define the overall structure of the MapReduce application by specifying both the Mapper and Reducer classes. If we want a map-only job, we omit the definition of the Reducer class and set the number of reduce tasks to zero: `job.setNumReduceTasks(0)`

Indicate the HDFS path for the application's output as well as the format of the data. Both the key and value formats need to be identified.

We run the job and wait. The driver waits at this point until the `waitForCompletion` function returns.

Fouad DAHAK
ESIA, 2024/2025 ensia

Map Reduce Applications

Mapper Implementation

Section 04 41

The CSVParser class isn't a standard Hadoop class, but we use it to simplify the parsing of CSV files.

```
import java.io.IOException;
import au.com.bytecode.opencsv.CSVParser;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class FlightsByCarrierMapper extends
Mapper<LongWritable, Text, Text, IntWritable> {

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        if (key.get() > 0) {
            String[] lines = new CSVParser().parseLine(value.toString());
            context.write(new Text(lines[8]), new IntWritable(1));
        }
    }
}
```

The specification of the Mapper class explicitly identifies the formats of the key/value pairs that the mapper will input and output.

The Mapper class has a single method, named map. The map method names the input key/value pair variables and the Context object, which is where output key/value pairs are written.

The block of code in the if statement is where all data processing happens. We use the if statement to indicate that we don't want to parse the first line in the file, because it's the header information describing each column. It's also where we parse the records using the CSV Parser class's parseLine method.

With the array of strings that represent the values of the flight record being processed, the ninth value is returned to the Context object as the key. This value represents the carrier that completed the flight. For the value, we return a value of one because this represents one flight.

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Applications

Reducer Implementation

Section 04 42

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class FlightsByCarrierReducer extends
Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    protected void reduce(Text token, Iterable<IntWritable> counts,
        Context context) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable count : counts) {
            sum += count.get();
        }
        context.write(token, new IntWritable(sum));
    }
}
```

The specification of the Reducer class explicitly identifies the formats of the key/value pairs that the reducer will input and output.

The Reducer class has a single method, named reduce. The reduce method names the input key/value pair variables and the Context object, which is where output key/value pairs are written.

The block of code in the for loop is where all data processing happens. Remember that the reduce function runs on individual keys and their lists of values. So, for the particular key, the for loop sums the numbers in the list, which are all ones. This provides the total number of flights for the particular carrier.

This total is written to the context object as the value, and the input key, named token, is reused as the output key.

Fouad DAHAK
ESIA, 2024/2025

Map Reduce Applications

Running The Job

Section 04 43

- 1 Compile the job code
`javac -classpath $CLASSPATH MapRed/FlightsByCarrier/*.java`
- 2 Build a JAR file
`jar cvf FlightsByCarrier.jar *.class`
- 3 Run the driver application
`hadoop jar FlightsByCarrier.jar FlightsByCarrier /user/root/airlinedata/2008.csv /user/root/output/flightsCount`
- 4 Show the job's output file from HDFS
`hadoop fs -cat /user/root/output/flightsCount/part-r-00000`

Fouad DAHAK
ESIA, 2024/2025

Chapter II : Big Data Ecosystem & Technologies

Summary and Q&A

44

Thank You

Q & A

Fouad DAHAK
ESIA, 2024/2025